A Hash Table Construction Algorithm for Spatial Hashing Based on Linear Memory

Cesar Tadeu Pozzer
University of Skövde - HiS
Box 408, 541 28
Skövde - Sweden
pozzer@inf.ufsm.br

Cícero A. de Lara Pahins
Federal University of Santa Maria - UFSM
Santa Maria, RS - Brazil
cicerolp@inf.ufsm.br

Ilona Heldal
University of Skövde - HiS
Box 408, 541 28
Skövde - Sweden
ilona.heldal@his.se

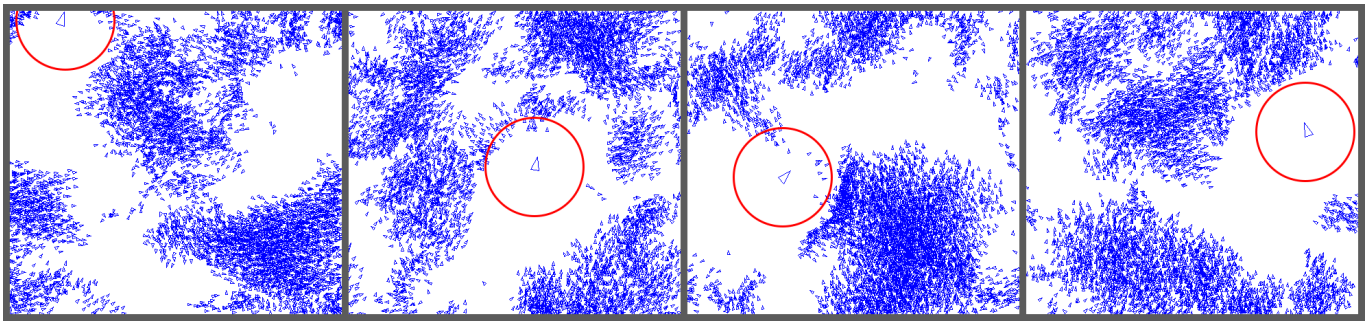


Figure 1. Steering behavior simulation.

ABSTRACT

Spatial hashing is an efficient technique to speed up proximity queries on moving objects in the space domain, suitable for computer entertainment applications and simulations. This paper presents an efficient three-step algorithm for building a 1D hash table for spatial hashing needed to perform fast queries on objects for location and proximity detection. In contrast to existing solutions, this algorithm uses fixed-size vectors and pivots instead of dynamic data structures to deal with collisions in the hash table. This also enables iterating through entities and performing proximity queries in a linear memory. Experiments conducted shows that the proposed algorithm is, on average, at least 3 times faster than existing solutions based on dynamic data structures. This contributes to realizing interactive frame rates with massive number of moving entities.

Author Keywords

Spatial Hashing; Space Partitioning; Data Structure; Steering Behaviors; Artificial Intelligence

ACM Classification Keywords

E.2. Data: Data Storage Representations

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACE '14, November 11 - 14 2014, Funchal, Portugal
Copyright © 2014 ACM 978-1-4503-2945-3/14/11...$15.00
http://dx.doi.org/10.1145/2663806.2663862

INTRODUCTION

As computers gain performance, more complex and large simulations are possible. When the real-time aspect is crucial, such as for digital games, faster algorithms become necessary for providing adequately fast response times.

Spatial hashing is a well-known technique to speed up the proximity query operations, commonly found in simulations composed by a large number of entities [5]. It maps 2D or 3D objects positions into a 1D hash table by using a hash function, where the environment is divided into a set of connected cells (lattice), forming a spatial grid.

This technique is a result of meeting growing demands on computation resources needed to process large simulations, composed by a large number of moving individuals, since it allows to perform most query operations in O(1) time complexity [4]. As presented by Hastings [3], spatial hashing can be used to search faster for entities or to optimize a number of operations. These can be applied in digital games or simulations for object-object collision detection, object-terrain collision, view frustum, picking and AI decisions. Proximity queries are commonly used for many different applications to determine the relative configuration of two objects. Steering behaviors [7] is a typical example for that (where naïve algorithms are of $O(n^2)$ time complexity) and may benefit from an efficient spatial hashing solution.

There are many approaches to build up hash tables, considering also varying amount of objects during the simulation. In this paper, we propose a new, simple to implement, efficient[1] approach with low memory requirements for building the 1D

[1] In particular, in terms of temporal scalability.

hash table for spatial hashing. Instead of using dynamic data structures for handling hash collisions, as presented in [1, 3], our solution achieves better performance by using static data structures, which also allows the inclusion and removal of individuals without compromising performance. This algorithm is based on a simple strategy of rebuilding the 1D hash table for each simulation step as Hastings [3] and not using the incremental and more complex approach proposed by Buckland [1].

In order to demonstrate the effectiveness of this approach two experiments were performed. The first experiment shows the feasibility and advantages of implementing this algorithm using static data structures, while the second measures its effectiveness in a practical steering behavior scenario (Figure 1). The results indicate better performance (3-10 times), and also improved linear scalability compared to the solutions presented by Buckland [1] and Hastings [3], the two most recent hash table implementations.

RELATED WORK

There are various ways to improve performance and scalability of building dynamic hash tables by using parallel approaches, such as using modern GPUs (e.g., [2, 6]) or employing heuristics and other means to improve the data structure (e.g., [1, 3]). Passos [6] proposed an efficient solution, based on proprietary technology and complex data structure devised for GPUs, being able to sustain interactive frame rate to a massive number of moving entities. Buckland [1] and Hastings [3] use a similar solution to manage collisions in the hash table. They implement a dynamic list for each cell, as presented in Figure 2, where the hash size entries is equal with the number of cells of the spatial grid.

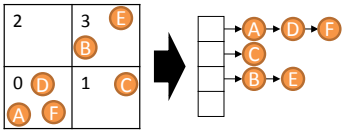


Figure 2. Example of a 2x2 (4 cells) grid mapped to a hash table representing 6 entities.

Buckland [1] also proposes an algorithm that compares the old and the current hash table for each entity and, if necessary, updates the corresponding list. The good performance of this solution is only guaranteed for infrequent updates. Hastings suggests an algorithm that wipes out every cell list for each update [3]. However, for both cases, many dynamic and also time consuming memory operations must be performed.

PROPOSED SOLUTION

Our method uses fixed-size vector and pivots to deal with collisions in the hash table, as shown in Figure 3, and, therefore, it is possible to compute the memory requirements of the data structure required to store the result of the algorithm *priori*. The use of pivots allows quick updates and operations to query the number of elements inside each cell. Further, it also allows iterating through entities and cells in a linear memory to perform proximity queries.

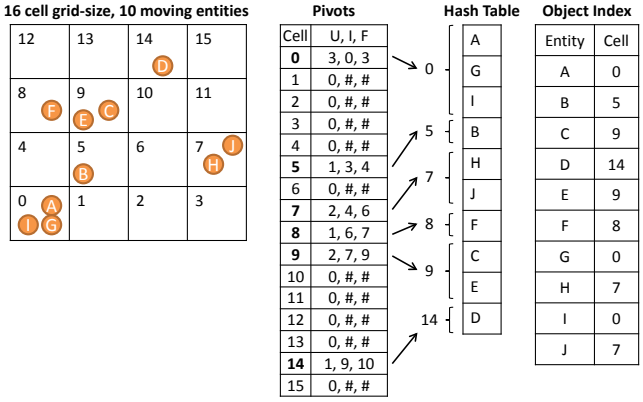


Figure 3. A 10 entity set is mapped into a 10 position, one-dimensional hash table statically sized to fit the exact number of individuals.

A three-step approach is proposed, as presented in Figure 4. First, the hash index is calculated for each entity (stored in the Object Index table) and hash table usage is computed for each cell (field usage (U) on table Pivots). Second, based on the number of elements each cell has, initial (I) and final (F) indexes for each cell are calculated (Pivot table). These lower and upper indexes represent the interval in the hash table reserved to store the entities for each non-empty cell. If the cell is empty, no space is reserved. Then, elements are distributed into the hash table based on these indexes. Figure 3 depicts the result of these operations. Having available pre-allocated space in the hash table, new entities can be inserted into the simulation without any dynamic memory operation. To remove entities, it is only necessary to adjust the indexes.

```
procedure SpatialHashingUpdate(SetOfEntities Entities)
    for e = 0, e < SizeOf(Entities), e = e + 1 do
        index = HashFunction(Entities[e])
        ObjectIndex[e] = index;
        Used[index] += 1;
    end for

    accum = 0
    for e = 0, e < SizeOf(Used), e = e + 1 do
        Initial[e] = accum
        accum += Used[e]
        Final[e] = accum
    end for

    for e = 0, e < SizeOf(Entities), e = e + 1 do
        HashTable[Final[ObjectIndex[e]] - 1] = Entities[e];
        Final[ObjectIndex[e]] -= 1;
    end for
end procedure
```

Figure 4. Definition of the update function. It is called every frame.

EXPERIMENTS AND RESULTS

To evaluate the performance of the proposed algorithm, two experiments were carried out. Results are presented in Table 1. All measures are shown in FPS (frames per second). The experiments consisted of a variable number of moving entities that are randomly placed in a 2-dimensional space. The grid size ranges from 4x4 up to 128x128 cells. B, H and P fields denote the performance for Buckland [1], Hastings [3] and our proposed method, respectively. All experiments were performed on a notebook PC with an Intel Core i7- 4700MQ

CPU, Turbo Boost enabled (clocked up to 3.4 GHz), 8 GB of RAM, running on a single thread.

The first experiment focused exclusively on the hash table construction time for different set sizes, ranging from 50k to 800k moving entities (Table 1(a)). Our solution demonstrated a better performance for all cases, scales in the same manner as Hastings [4] for each grid size (see Figure 5), and was the only algorithm able to sustain interactive frame rates over 400k entities. Further, as a consequence of the fast and fixed memory management, the variation across all tested space partition granularities was minimal, since the performance is not dependent on the number of collisions. Buckland [1] solution is the most sensitive algorithm to changes in the grid size while keeping the number of entities constant. Buckland [1] solution presents a performance gain when the space partition granularity is increased. This is due to a heavily branch dependent prediction algorithm implementation.

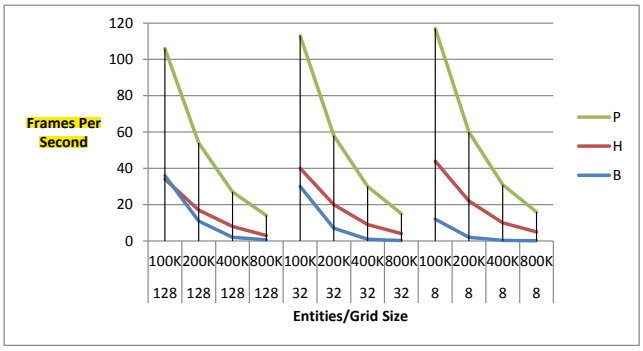


Figure 5. Performance analysis from Table 1(a).

The second experiment evaluates the algorithms in a steering behavior [7] setup, thus the reduced frames per second compared to the first experiment. This is a typical AI problem where proximity queries are used for selecting entities around each individual in order to select individual behaviors (Figure 1). For that, we adapted the AI framework developed by Buckland [1] (namely Steering Behaviors - Another Big Shoal) to enable all three algorithms running in the same conditions and optimizations. This framework offers a wide range of steering behavior simulations and settings, such as options to change the number of entities, the granularity of space partitioning, and the query radius.

In this simulation, we used the flocking behavior (defined by three simple steering forces: alignment, cohesion, and separation). Further, every moving entity should evade from a so called predatory entity (see Figure 1). Flocking behavior requires a demanding proximity query function (naively $O(n^2)$ time complexity). This operation can be significantly improved by using proximity queries based on spatial hash. Figure 6 shows the pseudocode of our proximity query function.

Table 1(b) shows the performance measures of the second experiment. In this experiment, the proximity query function required by the steering behaviors consumes a large portion of processing time, so it is important to be able to quickly

```
procedure TAGNEIGHBORS(Entity E, QueryRadius)
    RangedCells = CalculateCellsInRange(E, QueryRadius)

    for each cell in RangedCells
        for idx = Initial[cell], idx < Initial[cell] + Used[cell], idx += 1 do
            if SqDistance(HashTable[idx], E) < QueryRadius^2 then
                TagNeighbor(HashTable[idx])
            end if
        end for
    end for
end procedure
```

Figure 6. Definition of the proximity query function used by our method.

tag entities within a query radius and quickly iterate through entities stored in the cells of the space partition.

Note that the grid size is inversely proportional to the granularity of the cells. Smaller granularity leads to a costly and improved precision in the proximity queries. Our algorithm gains performance as the grid size increases. It is also important to notice that for small grid sizes (e.g., 4x4), the proposed solution performs similar to existing solutions, since the proximity query consumes a huge part of the processing time. However, since (a) the main purpose of the spatial hashing is to optimize proximity queries, and (b) performance increases as the grid size, there is little or no reason to use small grid sizes.

Performance Analysis

Regarding time complexity, our solution presents similar results when compared to Hastings and Buckland [1, 3]. However, our method uses a dense vector data structure representation and, thus, no memory is wasted with list linkage.

Applications may have a very dynamic behavior, presenting an unpredictable number of moving entities in a given time. In these cases, a memory reallocation may be too time consuming, so optimization is crucial. By using our pivot scheme, it is possible to allocate hash tables and entity vectors to support the biggest setup and, then, limit the usage in the spatial hashing update function (Figure 4). This solution offers an appealing performance, since the CPU cache automatically optimize memory operations to the adequate space usage and ignore unutilized spaces.

Our approach maintains the same performance regardless of whether the entities are moving or not. When compared against other (dynamic memory) solutions, such as [1, 3], our static memory solution provides a much desirable feature to real-time applications. Buckland [1] solution is the least efficient for moving entities.

CONCLUSIONS AND FUTURE WORK

In this paper, we present a compact, easy to implement and, efficient algorithm for building spatial hash tables to represent dynamic and moving entities. This solution proved to be faster when compared to other existing solutions.

Our major contributions have (i) shown that it is possible to compute the memory requirements of data structure required to store the result of the algorithm priori and (ii) demonstrated that this leads to a performance increase (of at least 3 times) over existing, dynamic data structure solutions.

		Grid Size																	
		128x128			64x64			32x32			16x16			8x8			4x4		
		B	H	P	B	H	P	B	H	P	B	H	P	B	H	P	B	H	P
Entities	50k	91	65	205	100	72	208	90	78	214	70	83	218	46	84	219	22	89	220
	100k	36	34	106	37	37	107	30	40	113	21	43	117	12	44	117	6	46	117
	200k	11	17	54	9	18	56	7	20	58	4	21	60	2	22	60	1	23	60
	400k	2	8	27	1	8	29	1	9	30	0.5	10	31	0.3	10	31	0.2	10	31
	800k	0.6	3	14	0.3	4	15	0.2	4	15	0.1	4	16	0.1	5	16	0.1	5	16

(a)

		Grid Size																	
		128x128			64x64			32x32			16x16			8x8			4x4		
		B	H	P	B	H	P	B	H	P	B	H	P	B	H	P	B	H	P
Entities	5k	7	156	225	35	223	266	92	235	347	140	166	206	107	71	123	50	26	46
	10k	3	59	81	16	84	124	38	79	156	42	43	63	25	15	33	10	5	11
	20k	1	21	31	7	26	44	13	30	53	11	10	22	5	3	8	2	1	2
	40k	0.8	5	10	2	7	13	3	7	13	2	2	5	1	0.8	1	0.6	0.2	0.5

(b)

Table 1. (a) Hash table construction time. (b) Results of practical AI application simulation including steering forces calculation and animation. All measures are shown in FPS (frames per second). The higher the value, the better the performance is.

Taking advantage of the performance given by our algorithm, developers can optimize a number of operations such as object-object collision detection, object-terrain collision, view frustum, picking and AI decisions, leading to much more appealing and seamless interactive entertainment applications and simulations.

It can be noticed that the pseudocode in Figure 4 can easily be parallelized. As future work, a parallel solution for multithreaded CPUs, as well experiments on parallel hardware found in commercial video cards should be performed.

ACKNOWLEDGMENTS

Cesar Tadeu Pozzer has been granted scholarship from CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico), CISB (Centro de Pesquisa e Inovação Sueco-Brasileiro), and Saab AB.

Cícero Augusto de Lara Pahins would like to thank CAPES-Brazil for financial support.

ADDITIONAL AUTHORS

Jonas Mellin (University of Skövde, Skövde - Sweden, jonas.mellin@his.se) and Per M. Gustavsson (Combitech/Saab, Box 46, 541 21, Skövde - Sweden, per.m.gustavsson@saabgroup.com).

REFERENCES

1. Buckland, M. *Programming Game AI by Example*, 1 ed. Wordware game developer's library. 2005.
2. Chiara, R. D., Erra, U., Scarano, V., and Tatafiore, M. Massive simulation using gpu of a distributed behavioral model of a flock with obstacle avoidance. In *Proceedings of the International Workshop on Vision, Modeling, and Visualization (VMV)* (2004), 233–240.
3. Hastings, E. J., Mesit, J., and Guha, R. K. Optimization of large-scale, real-time simulations by spatial hashing. In *Proceedings of the Summer Computer Simulation Conference*, vol. 37 (2005), 9–17.
4. Koutsiamanis, R.-A., and Efraimidis, P. S. An exact and o(1) time heaviest and lightest hitters algorithm for sliding-window data streams. In *Proceedings of the Fifth FTRA International Conference on Multimedia and Ubiquitous Engineering* (2011), 89–94.
5. Lefebvre, S., and Hoppe, H. Perfect spatial hashing. In *ACM SIGGRAPH* (2006), 579–588.
6. Passos, E. B., Joselli, M., Zamith, M., Clua, E. W. G., Montenegro, A., Conci, A., and Feijo, B. A bidimensional data structure and spatial optimization for supermassive crowd simulation on gpu. *Computer Entertainment 7*, 4 (2010), 1–15.
7. Reynolds, C. W. Interaction with groups of autonomous characters. In *Proceedings of Game Developers Conference*, CMP Game Media Group (2000), 449–460.